Assignment 1: MCP Fundamentals

Theory Questions

1. Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you'd reach for each.
2. A local MCP server usually talks to Claude Desktop over stdio instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?

FastMCP Documentation Questions

1. FastMCP supports three transports: stdio, http, and sse. Read the deployment docs and explain when you'd pick stdio vs http, and what actually changes in your `mcp.run()` call to switch between them.
2. FastMCP automatically turns some tool return values into "structured output" and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.

Practical Assignment: Design Your Own MCP Use Case

Pick a real problem, not a toy example, something you'd actually want an assistant to help with. Build a small FastMCP server around it with exactly 3 primitives:

- one **tool**, something the model should decide to call mid conversation (an action, takes input, does something, returns a result)
- one **resource**, addressable data the client can pull into context (not an action, no side effects, just "here's data at this URI")
- one **prompt**, a reusable instruction template a person selects on purpose

Deliverable

- `server.py` with all three, each with a clear docstring
- a short README explaining the use case in 2-3 sentences and why you picked a tool vs a resource vs a prompt for each piece, not just what they do

Submission Instructions

1. **Theory answers, LaTeX (10 marks)**: write up both theory questions and both FastMCP documentation questions in LaTeX, compile to PDF, submit the PDF.
2. **Implementation, GitHub link (10 marks)**: push your project to a GitHub repo with proper setup, `pyproject.toml` /dependency file, `.gitignore`, README, no `.env` or secrets committed. Submit the repo link.
3. **System design diagram and project structure (5 marks)**: include a diagram or flowchart showing how your tool, resource, and prompt fit together, plus a short explanation of your project structure (what each file does).
4. **Actual coding (15 marks)**: the server runs, all 3 primitives work as described, verified live in the MCP Inspector or Claude Desktop.
5. **Submit on time.** Late submissions will not be scored on the above breakdown.

Total: 40 marks

Sources

- Tools - FastMCP

deboistech · 2026-09-29

- FastMCP Server Class Overview

deboistech

deboistech · 2026-09-29